



Peirce's law is $((A \Rightarrow B) \Rightarrow A) \Rightarrow A$. It mentions only $\Rightarrow$, and still it needs the excluded third.

- (a) Under the flags $i_{ET} : \Pi\alpha : *. \alpha \vee \neg\alpha$, $A : *$, $B : *$, derive it in λC in flag format, naming the rule on every line; the derived rules of the box may be single steps. Hint: $i_{ET} A$ gives two cases; in the case $\neg A$, first build a proof of $A \Rightarrow B$.
- (b) Complete the Lean proof below with tactics, one tactic per line of your derivation. The excluded third is the hypothesis `em`.

```
theorem peirce (A B : Prop) (em : ∀ p : Prop, p ∨ ¬p) :
  ((A → B) → A) → A := by
  sorry
```

Nederpelt & Geuvers, *Type Theory and Formal Proof* (CUP, 2014), §7.4; Peirce 1885.

I Definitions

Flag format. A flag holds an assumption or a declaration, and the lines under its pole may use it. Each line names its rule and the lines or flags it uses. A declaration $x : A$ is a flag too, and *(abst)* closes it.

The codings. Nothing is added to λC ; each connective is a Π -type over all propositions $C : *$:

$$\perp \equiv \Pi\alpha : *. \alpha, \quad \neg A \equiv A \rightarrow \perp, \quad A \vee B \equiv \Pi C : *. (A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow C.$$

A proof of $A \Rightarrow B$ is a term of type $A \rightarrow B$. The excluded third is not derivable, so i_{ET} assumes it.

Derived rules, each a term built from the coding. The premisses are the flags or lines the term needs.

$$\frac{x : A}{\lambda C : *. \lambda f : A \rightarrow C. \lambda g : B \rightarrow C. f x : A \vee B} \text{ (}\vee\text{-intro-left-sec)} \quad \frac{u : \neg A \quad w : A}{u w : \perp} \text{ (}\neg\text{-elim)}$$

$$\frac{y : B}{\lambda C : *. \lambda f : A \rightarrow C. \lambda g : B \rightarrow C. g y : A \vee B} \text{ (}\vee\text{-intro-right-sec)} \quad \frac{f : \perp \quad C : *}{f C : C} \text{ (}\perp\text{-elim)}$$

$$\frac{u : A \vee B \quad C : * \quad v : A \rightarrow C \quad w : B \rightarrow C}{u C \vee w : C} \text{ (}\vee\text{-elim-sec)}$$

$\perp$ -elim and $\neg$ -elim are each one (*appl*); $\vee$ -elim-sec is three of them.

The two rules of λC you need. $B[x := N]$ is B with N substituted for x ; $A \rightarrow B$ is $\Pi x : A. B$ with x not in B .

$$\frac{\Gamma \vdash M : \Pi x : A. B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B[x := N]} \text{ (appl)} \quad \frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x : A. M : \Pi x : A. B} \text{ (abst)}$$

Application associates to the left.

I Tactics

A tactic script works from the goal upward: each tactic is one line of the derivation, read backward. The five you need, each with an example.

intro: the goal $A \rightarrow B$ raises a flag $a : A$; the goal becomes B .

```
example (p q : Prop) (hq : q) : p → q := by
  intro hp
  exact hq
```

exact: close the goal with a term that has its type.

```
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  exact (hp, hq)
```

apply: use $h : A \rightarrow B$ backward; the goal B becomes A .

```
example (p q r : Prop) (f : q → r) (g : p → q) (hp : p) : r := by
  apply f
  apply g
  exact hp
```

rcases: $\vee$ -elimination, one goal per case, with the names you give.

```
example (p q : Prop) (h : p ∨ q) : q ∨ p := by
  rcases h with hp | hq
  · right; exact hp
  · left;  exact hq
```

absurd: from a and $\neg a$, anything ($\neg$ -elim then $\perp$ -elim).

```
example (p q : Prop) (hp : p) (np : ¬p) : q := by
  exact absurd hp np
```

A bullet `·` starts the script for one goal; the hypothesis `em` is used as `em A : A ∨ ¬A`.

Name _____

Student ID _____

Answer

Out of room? Continue on the back of the page.